

Skill registry — 2026-05-25

A register of every custom skill I've written for Claude Code (Anthropic's coding assistant CLI), and whether it has measured data behind its claims yet. A "skill" here is a reusable slash command — a named recipe Claude Code runs when I type `/audit` or similar. 4 repos, 35 skills, 34 of them A/B-tested for token savings so far.

A/B-MEASURED SAVE RATES BY PORTFOLIO

+23% +78K tokens across 8 skills

AudiobookMaker (arm A 342K → arm B 264K)



+21% +27K tokens across 4 skills

mikkonumminen.dev (arm A 128K → arm B 101K)



+20% +170K tokens across 11 skills

Spacepotatis (arm A 853K → arm B 683K)



+7% +40K tokens across 11 skills

claude-skills (arm A 590K → arm B 550K)



Aggregate: +17% (+316K tokens across 34 skills). Save rate = (arm A – arm B) / arm A, summed per portfolio. Negative means the skill costs MORE per use than going cold — those skills encode rigor (audit thoroughness, protocol discipline, spec depth), not scout-savings. The 3× heuristic baseline assumed everything would save ~67%; measurement says the truth varies from +52% to –112% depending on what the skill encodes.

Per-skill, four numbers per row: **measured cost / use** (real tokens billed when a skill ran), **measured save / use** (A/B test: arm A minus arm B), **estimated cost / use** (the author's pre-measurement guess), and **estimated save / use** (the 3× heuristic applied to the estimated cost). Green = measured. Italic gray = estimated. Orange measured-save means the skill cost MORE in the A/B than the unstructured baseline — real findings, not bugs. Cost-or-save dashes (—) mean no data exists for that cell yet.

Per-repo summary

REPO	SKILLS	COST / USE MEASURED	SAVE / USE MEASURED	ESTIMATE-ONLY
AudiobookMaker	8	8	8	0
Spacepotatis	12	11	11	1
claude-skills	11	11	11	0
mikkonumminen.dev	4	4	4	0

All measurement counts are per single invocation of the skill. Cost-measured rows have a real run behind their per-use cost (a transcript-attributed session, or an A/B calibration arm-B). Save-measured rows have a real A/B test behind their per-use savings (calibration). Estimate-only rows have neither yet.

Claude Code built-ins

reference — not part of the portfolio

Built-in slash commands. Per-use cost is measured from real session transcripts the same way as the custom-skill rows. Per-use save is measured by treating the built-in's prompt as an arm-B recipe in the same A/B methodology used on custom skills — both measured columns are real numbers, not heuristics. The estimated-cost cell stays "—" because built-ins have no SKILL.md author who wrote a guess; the estimated-save cell shows the project-wide 3× baseline heuristic prediction (2× measured cost) so the row carries the same heuristic-vs-measured contrast the custom-skill rows show. Methodology and raw arm-A / arm-B numbers: docs/audits/builtin-review-calibration-2026-05-22.md .

SKILL	STATUS	COST / USE <i>measured</i>	SAVE / USE <i>measured</i>	COST / USE <i>estimated</i>	SAVE / USE <i>estimated</i>
/review Claude Code built-in PR code review	✓MEASURED last 2026-05-21	1.05M tokens / use	46K tokens / use (63%)	—	2.10M tokens / use (3× heuristic)

Calibration honesty — where my guesses landed

Here are the rows where I had a guess before I had data. The “How wrong” column is the measurement divided by the guess. Green means I landed within $\pm 10\%$. Orange means I was off by 5 $\times$ or more in either direction. 17 of 32 rows are orange. The fix is not to write better guesses next time. The fix is to keep measuring.

SKILL	MY GUESS (PER USE)	MEASURED (PER USE)	HOW WRONG
AUDIOBOOKMAKER release-cut	2K	188K	111 $\times$ under
SPACEPOTATIS equipment	4K	401K	93 $\times$ under
CLAUDE-SKILLS mikko-help	7K	644K	92 $\times$ under
CLAUDE-SKILLS session-cost	8K	409K	51 $\times$ under
SPACEPOTATIS modular-architecture-audit	5K	129K	26 $\times$ under
AUDIOBOOKMAKER release-bundle-audit	4K	87K	23 $\times$ under
AUDIOBOOKMAKER worktree-launch	1K	23K	18 $\times$ under
AUDIOBOOKMAKER work-session	2K	23K	14 $\times$ under
AUDIOBOOKMAKER ci-failure-triage	2K	25K	13 $\times$ under
SPACEPOTATIS new-enemy	6K	68K	12 $\times$ under
SPACEPOTATIS new-story	5K	56K	10 $\times$ under

SKILL	MY GUESS (PER USE)	MEASURED (PER USE)	HOW WRONG
AUDIOBOOKMAKER pronunciation-corpus-add	2K	17K	9.5× under
CLAUDE-SKILLS mikko-install	4K	35K	8.8× under
SPACEPOTATIS new-mission	8K	50K	6.2× under
CLAUDE-SKILLS skill-usage	4K	25K	6.2× under
SPACEPOTATIS new-perk	9K	56K	6.2× under
CLAUDE-SKILLS skill-calibration	140K	27K	5.1× over
SPACEPOTATIS new-solar-system	13K	60K	4.6× under
AUDIOBOOKMAKER copyright-scan	3K	14K	4.5× under
AUDIOBOOKMAKER voice-pack-finnish	6K	27K	4.3× under
MIKKONUMMINEN.DEV skill-localUpdate	83K	21K	3.9× over
SPACEPOTATIS content-audit	15K	53K	3.5× under
SPACEPOTATIS new-migration	7K	21K	2.9× under
CLAUDE-SKILLS ai-codegen-smell-audit	75K	199K	2.7× under
SPACEPOTATIS balance-review	14K	33K	2.5× under

SKILL	MY GUESS (PER USE)	MEASURED (PER USE)	HOW WRONG
CLAUDE-SKILLS readme-drift-sync	40K	63K	1.6× under
SPACEPOTATIS save-roundtrip-audit	12K	19K	1.5× under
CLAUDE-SKILLS audit	425K	553K	1.3× under
MIKKONUMMINEN.DEV md-to-pdf	25K	21K	1.2× over
MIKKONUMMINEN.DEV sync-readmes	141K	119K	1.2× over
MIKKONUMMINEN.DEV skill-registry	130K	148K	1.1× under
CLAUDE-SKILLS react-anti-patterns-audit	23K	22K	1.0× within $\pm 10\%$

Multi-model calibration — does the save rate hold across models?

Same A/B methodology as the per-repo Sonnet calibrations, repeated with sub-agents dispatched as Opus and (later) Haiku. Each skill below was measured at least twice — once per model. The save rate is what changes: Opus is more efficient cold, so the recipe's value compresses; Sonnet rewards scaffolding skills more. Sorted by spread across models — biggest model-sensitivity first.

SKILL	SONNET	OPUS	HAIKU
CLAUDE-SKILLS readme-drift-sync	−112% arm A 30K → B 63K −33K saved/use	−4% arm A 54K → B 56K −2K saved/use	−6% arm A 41K → B 44K −3K saved/use
CLAUDE-SKILLS skill-usage	+53% arm A 49K → B 23K 26K saved/use	−14% arm A 32K → B 37K −5K saved/use	+57% arm A 64K → B 28K 36K saved/use
AUDIOBOOKMAKER copyright-scan	+49% arm A 65K → B 33K 32K saved/use	+16% arm A 48K → B 40K 8K saved/use	+57% arm A 61K → B 27K 35K saved/use
CLAUDE CODE BUILT-IN /review	+63% arm A 72K → B 26K 46K saved/use	+39% arm A 59K → B 35K 23K saved/use	+52% arm A 58K → B 28K 30K saved/use
CLAUDE-SKILLS audit	+2% arm A 117K → B 114K 3K saved/use	+2% arm A 214K → B 209K 5K saved/use	+23% arm A 92K → B 71K 21K saved/use
MIKKONUMMINEN.DEV skill-localUpdate	+52% arm A 44K → B 21K 23K saved/use	+51% arm A 59K → B 29K 30K saved/use	+33% arm A 46K → B 31K 15K saved/use

Each cell shows the A/B save rate (% of arm A tokens), the raw arm-A → arm-B tokens, and the absolute saved-per-use. Orange = the skill cost MORE on this model than the unstructured baseline. Sample size $N=1$ per (skill, model) pair — trust direction and magnitude, not two-significant-digit precision. The cross-portfolio observation is that recipe value shrinks as model capability rises: skills that save 50%+ on Sonnet typically settle at 20-40% on Opus because the cold arm gets cheaper, not because the recipe arm gets more expensive.

Per-repo skill tables

One row per skill. Measured rows have a green wash; estimated rows are white. Four data columns per row, in pairs: **Cost / use** measured + estimated, then **Save / use** measured + estimated. Green numbers come from real runs (transcript or A/B). Italic gray numbers are author estimates. Orange numbers are real A/B measurements showing the skill cost MORE per use than the unstructured baseline. Dashes mean no data exists for that cell yet. Every figure is per single invocation — no annual projections anywhere in this document.

AudiobookMaker					
8 skills · 8 measured · github.com/MikkoNumminen/AudiobookMaker					
SKILL	STATUS	COST / USE <i>measured</i>	SAVE / USE <i>measured</i>	COST / USE <i>estimated</i>	SAVE / USE <i>estimated</i>
ci-failure-triage When CI fails on a release tag or PR, walk the failure modes in the right order against a recipe library built from the project's fix(ci) commit history.	✓MEASURED last 2026-05-23	25K tokens / use	19K tokens / use	2K tokens / use	4K tokens / use
copyright-scan Scan a git diff (staged by default, or a named revision range / file set) for accidental third-party copyright leaks before they land on origin.	✓MEASURED last 2026-05-12	14K tokens / use	32K tokens / use	3K tokens / use	6K tokens / use
pronunciation-corpus-add Append a Finnish pronunciation failure (a word the Chatterbox Grandmom voice mispronounces) to the project's pronunciation corpus at docs...	✓MEASURED last 2026-05-23	17K tokens / use	2K tokens / use	2K tokens / use	4K tokens / use
release-bundle-audit Audit the AudiobookMaker PyInstaller release bundle for unused dependencies, dead-code data files, and accidental ML-stack pollution; then propose spec-only fixes on a `chore/release-bundle-size` branch.	✓MEASURED last 2026-05-23	87K tokens / use	-9K tokens / use	4K tokens / use	8K tokens / use
release-cut Cut a new AudiobookMaker release (bump APP_VERSION, tag vX.Y.Z, push, verify CI lands SHA-256 in release notes AND sidecar). Use this ski...	✓MEASURED last 2026-05-12	188K tokens / use	6K tokens / use	2K tokens / use	3K tokens / use

voice-pack-finnish Build a Finnish voice pack from a source audio file end-to-end â€” chunked analyze with ECAPA diarization, transcript validation per speaker, branching to LoRA training (long sources) or few-shot ref-clip packaging (short sources), and ear-check by synth.	✓MEASURED last 2026-05-23	27K tokens / use	29K tokens / use	6K tokens / use	12K tokens / use
work-session Start, pause, or finish a TODO.md work session as one of the 4 permanent Claude sessions in AudiobookMaker. Use this whenever the user sa...	✓MEASURED last 2026-05-23	23K tokens / use	-2K tokens / use	2K tokens / use	3K tokens / use
worktree-launch Start a new parallel Claude session safely.	✓MEASURED last 2026-05-23	23K tokens / use	126 tokens / use	1K tokens / use	3K tokens / use

Spacepotatis

12 skills · 11 measured · github.com/MikkoNumminen/Spacepotatis

SKILL	STATUS	COST / USE <i>measured</i>	SAVE / USE <i>measured</i>	COST / USE <i>estimated</i>	SAVE / USE <i>estimated</i>
balance-review Diff uncommitted changes to game data (weapons, enemies, waves, missions, perks, augments, loot pools, solar systems) and report DPS, TTK, energy-cost-per-DPS, augment-folded effective DPS, loot-pool roster shifts, and drop-rate deltas vs HEAD.	✓MEASURED last 2026-05-22	33K tokens / use	13K tokens / use	14K tokens / use	27K tokens / use
content-audit Pre-commit content invariants check â€” orphan refs (enemy / weapon / sprite / pod / loot-pool / mission system / story trigger), missing sprite generators, perk drop-weight sanity, mission prereq DAG, story integrity (voice + music files, trigger refs), storyTriggers helper coverage.	✓MEASURED last 2026-05-03	53K tokens / use	22K tokens / use	15K tokens / use	30K tokens / use
equipment Create, modify, or remove a weapon or piece of equipment (augment / reactor / shield / armor) â€” including visual changes to how it appears in combat or in the loadout UI.	✓MEASURED last 2026-05-03	401K tokens / use	-5K tokens / use	4K tokens / use	9K tokens / use

modular-architecture-audit Orchestrates the multi-phase modular-architecture audit + refactor.	✓MEASURED last 2026-05-22	129K tokens / use	46K tokens / use	5K tokens / use	10K tokens / use
new-enemy Scaffold a new enemy â€” enemies.json entry, BootScene placeholder sprite generator, optional test wave, and integrity test verification.	✓MEASURED last 2026-05-22	68K tokens / use	7K tokens / use	6K tokens / use	11K tokens / use
new-migration Add a Postgres schema migration end-to-end â€” dated SQL file in db/migrations/, Database interface update in src/lib/db.ts, prod applica...	✓MEASURED last 2026-05-22	21K tokens / use	4K tokens / use	7K tokens / use	14K tokens / use
new-mission Scaffold a new combat mission across missions.json, waves.json, the galaxy planet binding, and a smoke test â€” in one shot.	✓MEASURED last 2026-05-22	50K tokens / use	29K tokens / use	8K tokens / use	16K tokens / use
new-perk Scaffold a new mission-only perk â€” perks.ts entry, BootScene icon generator, HUD chip wiring, and (for actives) a switch case in PerkCo...	✓MEASURED last 2026-05-22	56K tokens / use	27K tokens / use	9K tokens / use	18K tokens / use
new-solar-system Add a new solar system to the galaxy â€” solarSystems.json entry (incl. galaxy bed), SolarSystemId union extension, optional unlock gatin...	✓MEASURED last 2026-05-22	60K tokens / use	6K tokens / use	13K tokens / use	26K tokens / use
new-story Add, modify, or remove story content â€” cinematic popups, voiceovers, music beds, body text, written synopses, and auto-trigger wiring.	✓MEASURED last 2026-04-30	56K tokens / use	14K tokens / use	5K tokens / use	11K tokens / use
new-weapon <i>Superseded by /equipment.</i>	REDIRECT	—	—	—	—
save-roundtrip-audit Pre-commit save-pipeline invariants check â€” walks every StateSnapshot field through the 8 layers of the save round-trip (snapshot inter...	✓MEASURED last 2026-05-03	19K tokens / use	7K tokens / use	12K tokens / use	24K tokens / use

SKILL	STATUS	COST / USE <i>measured</i>	SAVE / USE <i>measured</i>	COST / USE <i>estimated</i>	SAVE / USE <i>estimated</i>
ai-codegen-smell-audit Read-only audit for specific failure modes that recur in LLM-generated code â€” defensive guards on impossible cases, generic names in domain code, swallowed errors, single-use helpers, mirror tests, and so on.	✓MEASURED last 2026-05-17	199K tokens / use	76K tokens / use	75K tokens / use	150K tokens / use
audit Run a multi-phase robustness audit of the current codebase.	✓MEASURED last 2026-05-20	553K tokens / use	3K tokens / use	425K tokens / use	850K tokens / use
mikko-audit-suite Run every `mikko-*` audit skill that's relevant to the current codebase, in a sensible order, and produce one index document linking to each audit's report.	✓MEASURED last 2026-05-23	24K tokens / use	-4K tokens / use	—	—
mikko-help List every installed `mikko-*` skill with its description (or with its `barney:` field when `--barney` is passed).	✓MEASURED last 2026-05-20	644K tokens / use	-5K tokens / use	7K tokens / use	14K tokens / use
mikko-install Install, update, list, or uninstall `mikko-*` skills from the cloned `claude-skills` source repo into the user-wide skill directory (`~/...`	✓MEASURED last 2026-05-21	35K tokens / use	-6K tokens / use	4K tokens / use	8K tokens / use
react-anti-patterns-audit Read-only audit for six concrete React anti-patterns that recur in component code (key=index in lists, useEffect for derived state, dep-array lies, state mutation instead of replace, missing effect cleanup, multiple sources of truth).	✓MEASURED last 2026-05-23	22K tokens / use	-5K tokens / use	23K tokens / use	45K tokens / use
readme-drift-sync Detect drift between what the repo actually contains and what `README.md` claims, then rewrite ONLY the drifted sections in the README's ...	✓MEASURED last 2026-05-23	63K tokens / use	-33K tokens / use	40K tokens / use	80K tokens / use
security-audit Orchestrates the multi-phase security audit + remediation.	✓MEASURED last 2026-05-22	112K tokens / use	-4K tokens / use	—	—

session-cost Measure the token cost of a single Claude Code session — main thread plus every sub-agent it dispatched.	✓MEASURED last 2026-05-21	409K tokens / use	5K tokens / use	8K tokens / use	16K tokens / use
skill-calibration A/B-test a Claude Code skill against an unstructured baseline to measure actual token savings.	✓MEASURED last 2026-05-23	27K tokens / use	−12K tokens / use	140K tokens / use	280K tokens / use
skill-usage Measure actual Claude Code skill usage from local transcript JSONL files.	✓MEASURED last 2026-05-21	25K tokens / use	26K tokens / use	4K tokens / use	8K tokens / use

mikkonumminen.dev 4 skills · 4 measured · github.com/MikkoNumminen/mikkonumminen.dev					
SKILL	STATUS	COST / USE <i>measured</i>	SAVE / USE <i>measured</i>	COST / USE <i>estimated</i>	SAVE / USE <i>estimated</i>
md-to-pdf Render a markdown report (or any HTML) to a styled PDF using the local Chrome's --print-to-pdf flag.	✓MEASURED last 2026-05-23	21K tokens / use	15K tokens / use	25K tokens / use	50K tokens / use
skill-localUpdate Refresh every local artifact the site renders about your portfolio skills — the JSON the /contact terminal fetches at runtime, the measurement-overlaid annual totals, and the downloadable skills-registry PDF — in one sequenced chain.	✓MEASURED last 2026-05-23	21K tokens / use	23K tokens / use	83K tokens / use	165K tokens / use
skill-registry Scan every sibling repo under D:/koodaamista for `.claude/skills/*/SKILL.md` files and emit a consolidated registry as a structured JSON ...	✓MEASURED last 2026-05-20	148K tokens / use	−6K tokens / use	130K tokens / use	260K tokens / use
sync-readmes Audit project data against sibling repos' READMEs and open a PR with drift corrections.	✓MEASURED last 2026-05-18	119K tokens / use	−4K tokens / use	141K tokens / use	282K tokens / use

Method, in plain language

How the numbers in this document are produced, what they mean, and where I am still guessing.

Per single use — the only unit on this page

Every number in the document is per one invocation of the skill. There are no annual figures, no monthly figures, no "tokens per year" anywhere. The original document had those; they were models stacked on guesses about how often a skill gets used. This version drops them. If you want to know what a skill costs you over a year, multiply the per-use number by however many times you'll actually run it — the model wasn't going to be more honest than that anyway.

How “measured cost / use” is produced

Two sources, both real. **Transcript measurement:** every Claude Code session writes a JSON-Lines transcript to `~/.claude/projects/<dir>/<sessionId>.jsonl`, with each assistant message carrying an `attributionSkill` field when a skill is active. The `skill-usage` skill walks those files, groups by skill, sums `input_tokens + output_tokens + cache_creation_input_tokens` (cache reads excluded — those are paid upstream), dedupes by `requestId`, and reports per-use averages across whatever invocations landed in the 90-day window. That's "what happened in production."

A/B calibration arm-B: when a skill hasn't been invoked in production (no transcript data), an A/B run still spends real tokens — a Sonnet sub-agent followed the `SKILL.md` end-to-end on a representative task, with usage billed through the harness. Arm-B IS a real cost-per-use measurement, just from a controlled run instead of in-the-wild use. Rows in this state show the cost with a "tokens / use (A/B-measured)" sub-label. Rows with both transcript and A/B data prefer the transcript number — it's what happened, not what's reproducible.

How “measured save / use” is produced

One source: a calibration A/B test. Two Sonnet sub-agents solve the same task in fresh sandboxed worktrees — arm A cold (no `SKILL.md` access), arm B following the skill. Save / use is arm-A tokens minus arm-B tokens for that one run. **N = 1 per skill, single data point.** A re-run would produce different absolute numbers for both arms; trust direction and rough magnitude, not two-significant-digit precision.

Some skills show negative save / use in orange. Those are real findings: the skill arm spent MORE tokens than the unstructured arm, because the skill encodes rigor (e.g. a full-CRUD lifecycle or a multi-phase audit) that the unstructured arm skipped. The skill's value is completeness, not token compression. The arm-A / arm-B numbers are preserved on each calibrated row's receipt for any downstream consumer that wants to see both sides.

How “estimated cost / use” is produced

I made up the number. I imagined what running the skill should cost in tokens and wrote that down. There's no math behind these. They are guesses by the person who built the skill, written down before any measurement existed.

The estimates stay in the document so the picture covers every skill, not just the ones I've measured. Every measured estimate I had has turned out to be off, usually low by 5× to 100× — apply the same skepticism to the rows that haven't been measured yet. If anything, the un-measured estimates are likely to be *more* wrong: I measured the ones I felt most confident about first.

How “estimated save / use” is produced

3× heuristic on the estimated cost: *saved* $\approx$ 2× *cost*. The model says "doing this without the skill would cost about 3× what the skill costs," so the saved-per-use is 2× the cost-per-use. The 3× is a handful-of-side-by-side-runs guess; the May-2026 Spacepotatis calibration showed it's overstated by roughly 3× at the portfolio level (measured savings averaged ~22% of arm-A cost, vs the heuristic's ~67%). Treat estimated savings as a possibly-too-optimistic upper bound.

Cost is on both sides of the underlying subtraction, so the estimated save figure is more sensitive to bad cost estimates than the estimated cost is. An italic estimated-save stacked on top of an italic estimated-cost is a model on top of a guess — least trustworthy cell on the page. A green measured-save on top of a green measured-cost is the most trustworthy. The visual treatment matches that hierarchy.

What this document does NOT claim

- No production cost numbers. This is development-time tooling on my own machine.
- No comparison to other engineers' setups. I only have transcripts for one engineer.
- No per-feature ROI claim. This is per-skill cost, not per-feature value.

- No assertion that any of this scales linearly to a team. It might. I have not measured.
- No claim that the modeled savings would survive an actual A/B test against well-prompted unstructured chats. They might; I haven't tested.

Reproducibility

From inside this repo (`mikkonumminen.dev/`) on a machine with Claude Code installed:

1. `/mikko-skill-usage` — writes `.claude/agent-verdicts/SKILL-USAGE-LATEST.json` from local transcripts.
2. `/skill-localUpdate` (formerly `/skill-pdf`) — re-walks the portfolio inventory, applies the measurement overlay, renders this PDF.

The data the renderer reads is committed to the repo at `public/data/skills-registry.json` . If you want to inspect what's behind a number on this page, that file is the source of truth. The dated history lives in `.claude/agent-verdicts/SKILL-REGISTRY-*.json` .

The *last* `<date>` marker on a measured row is the timestamp of the most recent Claude Code transcript that invoked that skill. The transcript itself stays on my machine because it contains code and context from private repos. If you want to verify the measurement methodology rather than the measurement, the parser source is at `~/.claude/skills/mikko-skill-usage/SKILL.md` — it is a JSONL parser, not a network call to anything.